

Juniorprogrammierer.de

Go 1: Some new stuff

2024/25/26 – Sascha Stojanovic

Some more Go facts

- Today we will have some more known concepts easy to grasp
- We will also face a completely new concept
- In case our time is ending soon please feel free to explore go on your own

<https://www.youtube.com/watch?v=iDQAZEJK8II&list=PLoILbKo9rG3skRCj37Kn5Zj803hhiuRK6>

Functions

- Importing parameter in Functions are as in Java but returning parameter are special, you can return n values

```
func myFunction(x int, y string) (result int, txt1 string) {  
    result = x + x  
    txt1 = y + " World!"  
    return  
}  
  
func main() {  
    a, b := myFunction(5, "Hello")  
    fmt.Println(a, b)  
}
```

Struct

A struct (short for structure) is used to create a collection of members of different data types, into a single variable => In Java attribute of a class

```
type Person struct {  
    name string  
    age int  
    job string  
    salary int  
}  
  
func main() {  
    var pers1 Person  
    var pers2 Person  
  
    // Pers1 specification  
    pers1.name = "Hege"  
    pers1.age = 45  
    pers1.job = "Teacher"  
    pers1.salary = 6000  
}
```

Maps

- Maps are used to store data values in key:value pairs
- Create a map with values
 - `var a = map[string]string{"brand": "Ford", "model": "Mustang"}`
 - `b := map[string]int{"Oslo": 1, "Bergen": 2, "Trondheim": 3, "Stavanger": 4}`
- Create an empty map
 - `var a = make(map[string]string)`
 `a["brand"] = "Ford"`
 `a["model"] = "Mustang"`
- Create an empty map with and without make and do “`fmt.Println(x == nil)`” => what do you see?
- Check for an value or update an value of the map by using the key => `a["brand"] = "Sascha"`
- Delete a map value with delete function => `delete(a, "year")`
- Check for an value and for existenz => `val1, ok1 := a["brand"]`

Pointer (new) I

- Pointer => Share the position in Ram don't copy the value
 - &variable = Adresse von variable (Pointer erstellen)
 - *pointer = Wert an der Adresse (Pointer dereferenzieren)

```
func main() {  
    // Normale Variable  
    score := 100  
    fmt.Println("Start Score:", score)  
  
    // Pointer erstellen  
    scorePointer := &score  
    fmt.Println("Pointer Adresse:", scorePointer) // z.B. 0xc000014090  
    fmt.Println("Wert vom Pointer:", *scorePointer) // 100  
  
    // Wert über Pointer ändern  
    *scorePointer = 200  
    fmt.Println("Score nach Änderung:", score) // 200 - Original geändert!  
}
```

Pointer (new) I

```
// Funktion OHNE Pointer - bekommt eine Kopie
func addPointsCopy(points int) {
    points = points + 50
    fmt.Println(" Inside addPointsCopy:", points)
}

// Funktion MIT Pointer - bekommt das Original
func addPointsPointer(points *int) {
    *points = *points + 50
    fmt.Println(" Inside addPointsPointer:", *points)
}
```

```
func main() {
    score := 100
    fmt.Println("Start:", score)
    fmt.Println()

    // Ohne Pointer
    addPointsCopy(score)
    fmt.Println("Nach addPointsCopy:", score) // 100
    - NICHT geändert!
    fmt.Println()

    // Mit Pointer
    addPointsPointer(&score)
    fmt.Println("Nach addPointsPointer:", score) //
    150 - geändert!
}
```

Exercise

checkBalance function

- Takes the balance as a parameter
- Prints: "Balance: X €"

deposit function

- Takes a pointer to the balance and an amount
- Adds the amount to your balance
- Prints: "Deposited: X €"

withdraw function

- Takes a pointer to the balance and an amount
- If you have enough money:
 - Subtract the amount from your balance
 - Print: "Withdrawn: X €"
- If you DON'T have enough money:
 - Print: "Not enough money!"
- Don't change the balance

Expected Output:

Balance: 100 €
Deposited: 50 €
Balance: 150 €
Withdrawn: 30 €
Balance: 120 €
Not enough money!
Balance: 120 €

Important Hints:

- Pointers: When you need to CHANGE a value inside a function, use a pointer (*int)
- Calling with pointer: Use &myBalance to pass the memory address
- Using pointer value: Inside the function, use *balance to get/change the actual value
- Reading only: When you just want to READ a value (not change it), you don't need a pointer